

Preskilet video link:

<https://youtu.be/PxWw2PpNMSs>

---

## Cisco certificate



## Statement of Achievement

**Mayur Sabale**

has successfully achieved student level credential for completing the *Python Essentials 2* course, provided by Cisco Networking Academy in collaboration with OpenEDG Python Institute.

The graduate has studied:

- Design, develop, debug, execute, and refactor multi-module computer programs written in Python 3.
- Analyze and model real-life problems in OOP categories.
- Use the potential of Python in everyday applications, including IoT and DIY activities.
- Understand the role of a programmer in the software development process.
- Attempt the qualification PCAP – Certified Associate in Python Programming from OpenEDG Python Institute and continue their professional development at an advanced level.



Issued on: Apr 15, 2026



Scan to Verify

*Lynn Bloomer*

Lynn Bloomer  
Director, Cisco Networking Academy

# Lab Assignment Screenshot

1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

**Input Format:**

- The first input will be an integer  $n$ , the number of courses.
- The second input will be  $n$  integers representing the marks of the student in each of the  $n$  courses, separated by a space.

**Output Format:**

The student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

The student fails any course (marks < 40 in any course), print:

- "Fail".

**Note:**

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

passorFa...

```
1 n = int(input())
2 marks = input().split()
3
4 total = 0
5 fail = False
6
7 for i in range(n):
8     m = int(marks[i])
9     if m < 40:
10         fail = True
11         total += m
12
13 if fail:
14     print("Fail")
15 else:
16     percentage = total / n
17     print(f"Aggregate Percentage: {percentage:.2f}")
18
19 if percentage > 75:
20     print("Grade: Distinction")
21 elif percentage >= 60:
22     print("Grade: First Division")
23 elif percentage >= 50:
24     print("Grade: Second Division")
25 elif percentage >= 40:
```

CODETANTRA

Home Learn Anywhere

202501120047@mitaoe.ac.in Support Logout

12.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

**Input Format:**

- The input is an integer, representing the number of rows in the pattern.

**Output Format:**

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

**Note:**

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

numberP...

```
1 # Type Content here...
2
3 n = int(input())
4 for i in range(1,n+1):
5     for j in range(1,i+1):
6         print(j,end=" ")
7     print()
```

CODETANTRA

Home Learn Anywhere

202501120047@mitaoe.ac.in Support Logout

5.2.11. Scatter Plot for Age vs. Fare by Survived

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:

- Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
- Color the points based on the **Survived** column: **Red** for passengers who did not survive (**Survived = 0**). **Blue** for passengers who survived (**Survived = 1**).
- Set the title of the plot to "Age vs. Fare by Survival".
- Label the x-axis as 'Age' and the y-axis as 'Fare'.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name                                                 | Sex    | Age | SibSp | ParCh | Ticket    | Fare    | Cabin | Embarked |
|-------------|----------|--------|------------------------------------------------------|--------|-----|-------|-------|-----------|---------|-------|----------|
| 1           | 0        | 3      | Brund, Mr. Owen Harris                               | male   | 22  | 1     | 0     | A/5 21171 | 7.25    | S     |          |
| 2           | 1        | 1      | Cummings, Mrs. John Bradley (Florence Briggs Thayer) | female | 38  | 1     | 0     | PC 17599  | 71.2833 | C85   | C        |
| 3           | 1        | 3      | Heikkinen, Miss. Laina                               | female | 26  | 0     | 0     | STON/O2   | 3101282 | 7.925 | S        |
| 4           | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)         | female | 35  | 1     | 0     | 113803    | 53.1    | C123  | S        |
| 5           | 0        | 3      | Allen, Mr. William Henry                             | male   | 35  | 0     | 0     | 373450    | 8.05    | S     |          |

**Sample Data:**

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, "Brund, Mr. Owen Harris", male, 22, 1, 0, "A/5 21171", 7.25, S,
2, 1, 1, "Cummings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, "PC 17599", 71.2833, "C85", C
3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, "STON/O2", 3101282, 7.925, S
4, 1, 1, "Futrelle, Mrs. Jacques Heath (Lily May Peel)", female, 35, 1, 0, 113803, 53.1, "C123", S
5, 0, 3, "Allen, Mr. William Henry", male, 35, 0, 0, 373450, 8.05, S
```

AgeFareS...

```
1 # Import pandas as pd
2 # Import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Scatter Plot for Age vs. Fare by Survived
17
18
19 import pandas as pd
20 import matplotlib.pyplot as plt
21
22 # Load the Titanic dataset
23 data = pd.read_csv('Titanic-Dataset.csv')
24
25 # Data Cleaning
```

CODETANTRAHomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

5.2.1. Titanic Dataset

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:  
  
**Dataset Information:**  
The dataset is stored in a CSV file named `titanic.csv` and has been loaded using the `pandas` library. It contains the following columns:

- `Pclass`: Passenger class (1 = First, 2 = Second, 3 = Third).
- `Gender`: Gender of the passenger (male/female).
- `Age`: Age of the passenger.
- `Survived`: Survival status (0 = Did not survive, 1 = Survived).
- `Fare`: Ticket fare paid by the passenger.

  
**Visualization:**  
To represent these trends, you will create 5 visualizations using `Matplotlib`. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).  
  
**Visualization Details:**  
Write the code to create a series of visualizations as follows:  
**Bar Plot (Pclass Distribution):**

- Create a bar plot to show the distribution of passengers across the different passenger classes (`Pclass`).
- Use the color `skyblue` for the bars.

titanicDat...

123456789101112131415161718192021222324

```
1 ## Import pandas as pd
2 ## Import matplotlib.pyplot as plt
3
4 ## Load the Titanic dataset from the CSV file
5 df = pd.read_csv('titanic.csv')
6
7 ## Set up the figure for 5 subplots
8 fig, axes = plt.subplots(3, 2, figsize=(12, 12))
9
10 ## write the code..
11 import pandas as pd
12 import matplotlib.pyplot as plt
13
14 ## Load the Titanic dataset
15 df = pd.read_csv('titanic.csv')
16
17 ## Set up the figure with 3 rows and 2 columns
18 fig, axes = plt.subplots(3, 2, figsize=(12, 12))
19
20 ## 1. Bar Plot (Pclass Distribution)
21 pclass_counts = df['Pclass'].value_counts().sort_index()
22 axes[0, 0].bar(pclass_counts.index, pclass_counts.values,
23               color='skyblue')
24 axes[0, 0].set_title("Passenger Class Distribution")
```

TerminalTest cases

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.  
  
1. Get the number of survivors by gender (Sex).  
2. Get the number of non-survivors by gender (Sex).  
3. Get the number of survivors by embarkation location (Embarked\_S).  
4. Get the number of non-survivors by embarkation location (Embarked\_S).  
5. Calculate the percentage of children (Age < 18) who survived.  
6. Calculate the percentage of adults (Age >= 18) who survived.  
7. Get the median age of survivors.  
8. Get the median age of non-survivors.  
9. Get the median fare of survivors.  
10. Get the median fare of non-survivors.  
  
The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | ParCh | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

Sample Data:

titanicDat...

191192193194195196197198199200201

```
191 children = data[data['Age'] < 18]
192 survived_children_pct = children['Survived'].mean()
193 print(survived_children_pct)
194
195 ## 6. Calculate the percentage of adults (Age >= 18) who survived
196 adults = data[data['Age'] >= 18]
197 survived_adults_pct = adults['Survived'].mean()
198 print(survived_adults_pct)
199
200 ## 7. Get the median age of survivors
201 median_age_survivors = data[data['Survived'] == 1]['Age'].median()
```

Average time0.063 sMaximum time0.063 s1 out of 1 shown test case(s) passed

Test case 1Expected outputActual outputfemale: 233female: 233male: 109male: 109Name: Sex, dtype: int64Name: Sex, dtype: int64male: 468male: 468female: 81female: 81Name: Sex, dtype: int64Name: Sex, dtype: int64

TerminalTest cases

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.  
  
1. Calculate the survival rate by class.  
2. Calculate the survival rate by embarkation location (Embarked\_S).  
3. Calculate the survival rate by family size (FamilySize).  
4. Calculate the survival rate by being alone (IsAlone).  
5. Get the average fare by passenger class (Pclass).  
6. Get the average age by passenger class (Pclass).  
7. Get the average age by survival status (Survived).  
8. Get the average fare by survival status (Survived).  
9. Get the number of survivors by class (Pclass).  
10. Get the number of non-survivors by class (Pclass).  
  
The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | ParCh | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

Sample Data:

titanicDat...

201202203204205206207208209210211212213214215216217218219220221

```
201 ## 6. Get the average age by passenger class (Pclass)
202 avg_age_by_class = data.groupby('Pclass')['Age'].mean()
203 print(avg_age_by_class)
204
205 ## 7. Get the average age by survival status (Survived)
206 avg_age_by_survival = data.groupby('Survived')['Age'].mean()
207 print(avg_age_by_survival)
208
209 ## 8. Get the average fare by survival status (Survived)
210 avg_fare_by_survival = data.groupby('Survived')['Fare'].mean()
211 print(avg_fare_by_survival)
212
213 ## 9. Get the number of survivors by class (Pclass)
214 ## We filter for Survived == 1 and then use value_counts to match the
215 ## test case order
216 survivors_by_class = data[data['Survived'] == 1]
217 print(survivors_by_class)
218
219 ## 10. Get the number of non-survivors by class (Pclass)
220 ## We filter for Survived == 0 and then use value_counts to match the
221 ## test case order
222 non_survivors_by_class = data[data['Survived'] == 0]
223 print(non_survivors_by_class)
```

TerminalTest cases



CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

Sample Data:

titanicDat...

```
1 # import pandas as pd
2 # import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 ##
9 ## Load the Titanic dataset
10
11
12 # import pandas as pd
13 # import numpy as np
14
15 # Load the Titanic dataset
16 data = pd.read_csv('Titanic-Dataset.csv')
17
18 data['FamilySize'] = data['SibSp'] + data['Parch']
19
20 ## 1. Create a new column 'IsAlone'
21 data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
22
23 ## 2. Convert 'Sex' to numeric
24 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
25
```

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

Sample Data:

titanicDat...

```
1 # import pandas as pd
2 # import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # import pandas as pd
8
9 # Read the dataset
10
11 try:
12     file_name = input().strip()
13
```

Average time0.169 sMaximum time0.169 s1 out of 1 shown test case(s) passed

Test case 1Expected outputActual output

PassengerId Survived Pclass Fare Cabin Embarked

01037.2500

110571.2833

2107.9250

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

frequent...

```
1 # import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11
12 import pandas as pd
13 from itertools import combinations
14 from collections import Counter
15
16 # Prompt the user for the file name
17 file_name = input()
18
19 # Load the data
20 df = pd.read_csv(file_name)
21
22 # Group products by Date
23 grouped = df.groupby('Date')['Product'].apply(list)
24
25 # Store product pairs
26 pair_count = Counter()

```

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

Note:  
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1'''import pandas as pd
2
3## Prompt the user for the file name
4# file_name = input()
5
6## Load the data
7# df = pd.read_csv(file_name)
8
9## write the code..
10
11## Display the result
```

Average time0.017 sMaximum time0.036 s1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual output

sales\_data.csvsales\_data.csv

City sold the most products: Los AngelesCity sold the most products: Los Angeles

TerminalTest cases

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

Note:  
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
34# print(f"Total quantity sold: {total_quantity}")
35import pandas as pd
36
37## Prompt the user for the file name
38file_name = input()
39
40## Load the data
41df = pd.read_csv(file_name)
42## Group the data by 'Product' and sum the 'Quantity' for each
43... ## This creates a Series where index is Product and values are total quantities
```

Average time0.027 sMaximum time0.043 s1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual output

sales\_data.csvsales\_data.csv

Best selling product: Product ABest selling product: Product A

Total quantity sold: 23Total quantity sold: 23

TerminalTest cases

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

Note:  
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
34# print(f"Total quantity sold: {total_quantity}")
35import pandas as pd
36
37## Prompt the user for the file name
38file_name = input()
39
40## Load the data
41df = pd.read_csv(file_name)
42## Group the data by 'Product' and sum the 'Quantity' for each
43... ## This creates a Series where index is Product and values are total quantities
```

Average time0.027 sMaximum time0.043 s1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual output

sales\_data.csvsales\_data.csv

Best selling product: Product ABest selling product: Product A

Total quantity sold: 23Total quantity sold: 23

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1'''import pandas as pd
2
3## Prompt the user for the file name
4# file_name = input()
5
6## Load the data
7# df = pd.read_csv(file_name)
8
9
10## Find the product with the highest total quantity sold
11# best_product =
12# highest_quantity =
13
14## Display the result
15# print(f"Best selling product: {best_product}")
16# print(f"Total quantity sold: {highest_quantity}")
17'''
18## import pandas as pd
19
20## Prompt the user for the file name
21# file_name = input()
22
23## Load the data
24# df = pd.read_csv(file_name)
25
```

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
49# df = pd.read_csv(file_name)
50
51## Ensure 'Date' is in datetime format to extract the month easily
52df['Date'] = pd.to_datetime(df['Date'])
53
54## Calculate Sales for each row (Quantity * Price)
55df['Sales'] = df['Quantity'] * df['Price']
56
57## Group by Month (formatted as YYYY-MM) and sum the sales
58## We use .dt.to_period('M') to group by the specific month and year
59monthly_sales = df.groupby(df['Date'].dt.to_period('M'))['Sales'].sum()
```

Average time0.026 sMaximum time0.055 s1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 10.05 msDebugTest case 1Expected outputActual output

Expected outputActual output

Best month: 2025-01Best month: 2025-01

Total sales: \$1210.00Total sales: \$1210.00

TerminalTest cases

CODETANTRAHomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who not less than 40 marks in Subject 1: Count the number of

Sample Test Cases

Operatio...

```
1import numpy as np
2
3# = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5
6roll = a[:, 0]
7s1 = a[:, 1]
8s2 = a[:, 2]
9s3 = a[:, 3]
10
11# 1
12print("All student Details:\n", a)
13
14# 2
15print("Total Students:", a.shape[0])
16
17# 3
18print("All Student Roll Nos", roll)
19
20# 4
21print("Subject 1 Marks", s1)
22
23# 5
24print("Min marks in Subject 2", np.min(s2))
25
```

TerminalTest cases



CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

3.2.5. Numpy: Copying and Viewing Arrays

1234567891011121314151617181920212223

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original\_array and assigning it to view\_array.
- Creating a copy of the original\_array and assigning it to copy\_array.

After completing these steps, observe how modifying the view affects the original\_array, while modifying the copy does not.

**Input Format:**

- A single line of space-separated integers.

**Output Format:**

- After modifying the view:  
Original array after modifying view: <original\_array>  
View array: <view\_array>
- After modifying the copy:  
Original array after modifying copy: <original\_array>  
Copy array: <copy\_array>

Sample Test Cases

copyAnd...

1234567891011121314151617181920212223

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
18
19 # Modify the copy
20 copy_array[1] = 88
21 print("Original array after modifying copy:", original_array)
22 print("Copy array:", copy_array)
23
```

TerminalTest cases

PreviousResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operations

12345678910111213141516171819202122232425

You are given two arrays A and B. Your task is to complete the function array\_operations, which will convert these lists into NumPy arrays and perform the following operations:

- Arithmetic Operations:**
  - Compute the element-wise sum, difference, and product of the two arrays.
- Statistical Operations:**
  - Calculate the mean, median, and standard deviation of array A.
- Bitwise Operations:**
  - Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A<sub>1</sub> OR B<sub>1</sub>).

**Input Format:**

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

**Output Format:**

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

different...

12345678910111213141516171819202122232425

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     A = np.array(A)
7     B = np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A + B
11    diff_result = A - B
12    prod_result = A * B
13
14    # Statistical Operations
15    mean_A = np.mean(A)
16    median_A = np.median(A)
17    std_dev_A = np.std(A)
18
19    # Bitwise Operations
20    and_result = A & B
21    or_result = A | B
22    xor_result = A ^ B
23
24    # Output results with one space between each element
25    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
26    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
```

TerminalTest cases

PreviousResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

3.2.3. Numpy: Custom Sequence Generation

1234567891011121314

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

**Input Format:**

- The user will input three integer values: start, stop, and step, each on a new line.

**Output Format:**

- The program should print the generated sequence based on the input values.

Sample Test Cases

custom\$...

1234567891011121314

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9
10 sequence = np.arange(start, stop, step)
11
12 # Print the generated sequence
13 print(sequence)
14
```

TerminalTest cases

PreviousResetSubmitNext



CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

2.2.2. Captain of the Team

100%

🔍🔗🔧🔖

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

**Input Format:**  
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

**Output Format**  
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

captainof...

1

s = input()

2

3

nums = []

4

num = ""

5

6

for ch in s:

7

if ch != " ":

8

num += ch

9

else:

10

nums.append(int(num))

11

num = ""

12

13

nums.append(int(num))

14

15

tallest = nums[0]

16

17

for n in nums:

18

if n > tallest:

19

tallest = n

20

21

print(tallest)

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

2.2.1. Linear search Technique

100%

🔍🔗🔧🔖

Write a program to check whether the given element is present or not in the array of elements using linear search.

**Input format:**

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

**Output format:**

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print **Not found**.

**Sample Test Case:**  
Input:  
1 2 3 4 3 5 6  
3  
Output:  
2

Sample Test Cases

CTP1709...

1

arr = list(map(int, input().split()))

2

key = int(input())

3

4

found = False

5

6

for i in range(len(arr)):

7

if arr[i] == key:

8

print(i)

9

found = True

10

break

11

12

if not found:

13

print("Not found")

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (\*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 n = int(input())
2 for i in range(1,n+1):
3     print("*" * i)
```

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

HomeLearn Anywhere

202501120047@mitaoe.ac.inSupportLogout

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first  $n$  terms of the Fibonacci series.

Input Format:

- A single integer  $n$  representing the number of terms to generate.

Output Format:

- A single line containing the first  $n$  terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

fibonacci...

```
1 def fibonacci(n):
2     # write the code..
3
4     if n <= 1:
5         return 0
6     elif n == 2:
7         return 1
8     else:
9         return fibonacci(n-1) + fibonacci(n-2)
10
11
```

Average time0.015 s14.50 msMaximum time0.037 s37.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1DebugExpected output0 1 1 2 3Actual output0 1 1 2 3

Test case 2

TerminalTest cases

PrevResetSubmitNext